

Week 4 Exercises

Nischal Pant

May 19, 2022

Please complete all exercises below. You may use any library that we have covered in class. The data we will be using comes from the tidyr package, so you must use that.

- 1) Examine the who and population data sets that come with the tidyr library. the who data is not tidy, you will need to reshape the new_sp_m014 to newrel_f65 columns to long format retaining country, iso2, iso3, and year. The data in the columns you are reshaping contains patterns described in the details section below. You will need to assign three columns: diagnosis, gender, and age to the patterns described in the details.

Your tidy data should look like the following: country iso2 iso3 year diagnosis gender age count
1 Afghanistan AF AFG 1980 sp m 014 NA 2 Afghanistan AF AFG 1980 sp m 1524 NA 3 Afghanistan AF AFG 1980 sp m 2534 NA 4 Afghanistan AF AFG 1980 sp m 3544 NA 5 Afghanistan AF AFG 1980 sp m 4554 NA 6 Afghanistan AF AFG 1980 sp m 5564 NA

Details The data uses the original codes given by the World Health Organization. The column names for columns five through 60 are made by combining new_ to a code for method of diagnosis (rel = relapse, sn = negative pulmonary smear, sp = positive pulmonary smear, ep = extrapulmonary) to a code for gender (f = female, m = male) to a code for age group (014 = 0-14 yrs of age, 1524 = 15-24 years of age, 2534 = 25 to 34 years of age, 3544 = 35 to 44 years of age, 4554 = 45 to 54 years of age, 5564 = 55 to 64 years of age, 65 = 65 years of age or older).

Note: use data(who) and data(population) to load the data into your environment. Use the arguments cols, names_to, names_pattern, and values_to. Your regex should be = ("new_?(.)_(.)(.)")

<https://tidyr.tidyverse.org/reference/who.html>

```
#first lets select our packages that we will need for this lab
library(dplyr)
```

```
## Warning: package 'dplyr' was built under R version 4.2.3
```

```
##
```

```
## Attaching package: 'dplyr'
```

```
## The following objects are masked from 'package:stats':
```

```
##
```

```
## filter, lag
```

```
## The following objects are masked from 'package:base':
```

```
##
```

```
## intersect, setdiff, setequal, union
```

```
library(tidyr)
```

```
## Warning: package 'tidyr' was built under R version 4.2.3
```

```
library(ggplot2)
```

```
## Warning: package 'ggplot2' was built under R version 4.2.3
```

```
#lets load the data into our environment
```

```
data("who")
```

```
data("population")
```

```
#now we shall print the tibble and see what we are dealing with. We use
```

```
#colnames aswell to see our variables
```

```
print(who)
```

```
## # A tibble: 7,240 x 60
```

```
##   country iso2 iso3   year new_sp_m014 new_sp_m1524 new_sp_m2534 new_sp_m3544
```

```
##   <chr>   <chr> <chr> <dbl>         <dbl>         <dbl>         <dbl>         <dbl>
```

```
## 1 Afghani~ AF   AFG   1980           NA           NA           NA           NA
```

```
## 2 Afghani~ AF   AFG   1981           NA           NA           NA           NA
```

```
## 3 Afghani~ AF   AFG   1982           NA           NA           NA           NA
```

```
## 4 Afghani~ AF   AFG   1983           NA           NA           NA           NA
```

```
## 5 Afghani~ AF   AFG   1984           NA           NA           NA           NA
```

```
## 6 Afghani~ AF   AFG   1985           NA           NA           NA           NA
```

```
## 7 Afghani~ AF   AFG   1986           NA           NA           NA           NA
```

```
## 8 Afghani~ AF   AFG   1987           NA           NA           NA           NA
```

```
## 9 Afghani~ AF   AFG   1988           NA           NA           NA           NA
```

```
## 10 Afghani~ AF   AFG   1989           NA           NA           NA           NA
```

```
## # i 7,230 more rows
```

```
## # i 52 more variables: new_sp_m4554 <dbl>, new_sp_m5564 <dbl>,
```

```
## #   new_sp_m65 <dbl>, new_sp_f014 <dbl>, new_sp_f1524 <dbl>,
```

```
## #   new_sp_f2534 <dbl>, new_sp_f3544 <dbl>, new_sp_f4554 <dbl>,
```

```
## #   new_sp_f5564 <dbl>, new_sp_f65 <dbl>, new_sn_m014 <dbl>,
```

```
## #   new_sn_m1524 <dbl>, new_sn_m2534 <dbl>, new_sn_m3544 <dbl>,
```

```
## #   new_sn_m4554 <dbl>, new_sn_m5564 <dbl>, new_sn_m65 <dbl>, ...
```

```
print(population)
```

```
## # A tibble: 4,060 x 3
```

```
##   country      year population
```

```
##   <chr>        <dbl>      <dbl>
```

```
## 1 Afghanistan 1995    17586073
```

```
## 2 Afghanistan 1996    18415307
```

```
## 3 Afghanistan 1997    19021226
```

```
## 4 Afghanistan 1998    19496836
```

```
## 5 Afghanistan 1999    19987071
```

```
## 6 Afghanistan 2000    20595360
```

```
## 7 Afghanistan 2001    21347782
```

```
## 8 Afghanistan 2002    22202806
```

```
## 9 Afghanistan 2003    23116142
```

```
## 10 Afghanistan 2004    24018682
```

```
## # i 4,050 more rows
```

```
#now lets re-arrange and change a few elements of our data set. I will create
```

```
#a new data set which I will be using throughout the assignment
```

```
testwho <- who %>%
```

```
#we will use pipe operators and the pivot_longer function to
```

```
#reshape, anything that starts with new_ will be seperated into diagnosis
```

```
#gender, and age and our expression "new_?(.*)_(.)(.*)" will split
```

```
#them up into those colnames.
```

```
#values_to

pivot_longer(cols = starts_with("new_"),
  names_to = c("diagnosis", "gender", "age"),
  names_pattern = "new_(.*)_(.)(.*)",
  values_to = "count") %>%
select(country, iso2, iso3, year, diagnosis, gender, age, count)

#and using select we plug (those variables listed above) into testwho dataset

#now lets see the dataset
testwho
```

```
## # A tibble: 304,080 x 8
##   country    iso2 iso3   year diagnosis gender age   count
##   <chr>      <chr> <chr> <dbl> <chr>      <chr> <chr> <dbl>
## 1 Afghanistan AF    AFG   1980 sp        m     014    NA
## 2 Afghanistan AF    AFG   1980 sp        m    1524    NA
## 3 Afghanistan AF    AFG   1980 sp        m    2534    NA
## 4 Afghanistan AF    AFG   1980 sp        m    3544    NA
## 5 Afghanistan AF    AFG   1980 sp        m    4554    NA
## 6 Afghanistan AF    AFG   1980 sp        m    5564    NA
## 7 Afghanistan AF    AFG   1980 sp        m     65    NA
## 8 Afghanistan AF    AFG   1980 sp        f     014    NA
## 9 Afghanistan AF    AFG   1980 sp        f    1524    NA
## 10 Afghanistan AF    AFG   1980 sp        f    2534    NA
## # i 304,070 more rows
```

- 2) There are two common keys between the data sets, with who as the left table, join the population data by country and year so that the population is available within the who dataset.

```
#using our pipe operator and the left_join function we can join the
#two dataset, population and testwho by country and year

testwho <- testwho %>%
  left_join(population, by = c("country", "year"))
testwho
```

```
## # A tibble: 304,080 x 9
##   country    iso2 iso3   year diagnosis gender age   count population
##   <chr>      <chr> <chr> <dbl> <chr>      <chr> <chr> <dbl>      <dbl>
## 1 Afghanistan AF    AFG   1980 sp        m     014    NA        NA
## 2 Afghanistan AF    AFG   1980 sp        m    1524    NA        NA
## 3 Afghanistan AF    AFG   1980 sp        m    2534    NA        NA
## 4 Afghanistan AF    AFG   1980 sp        m    3544    NA        NA
## 5 Afghanistan AF    AFG   1980 sp        m    4554    NA        NA
## 6 Afghanistan AF    AFG   1980 sp        m    5564    NA        NA
## 7 Afghanistan AF    AFG   1980 sp        m     65    NA        NA
## 8 Afghanistan AF    AFG   1980 sp        f     014    NA        NA
## 9 Afghanistan AF    AFG   1980 sp        f    1524    NA        NA
## 10 Afghanistan AF    AFG   1980 sp        f    2534    NA        NA
## # i 304,070 more rows
```

- 3) Split the age column into two columns, min age and max age. Notice that there is no character separator. Check the documentation with `?separate` to understand other ways to separate the age column. Keep in

mind that 0 to 14 is coded as 014 (3 characters) and the other age groups are coded with 4 characters. 65 only has two characters, but we will ignore that until the next problem.

```
#to make things a little easier, first I will use an ifelse function
#to find values in testwho for variable age, if it is 014 to change it into
#0014 and then plug them back into the dataset
testwho$age <- ifelse(testwho$age == "014", "0014", testwho$age)
```

```
#and now using separate, I will split the age variable into min_age and
#max_age separated at the 3rd character which should split all the 4 char
#vales including 65 which is only 2 car
```

```
testwho <- testwho %>%
  separate(age,
    into = c("min_age", "max_age"),
    sep = c(2),
    convert = TRUE)
```

```
#now we check if it works
testwho
```

```
## # A tibble: 304,080 x 10
##   country iso2 iso3 year diagnosis gender min_age max_age count population
##   <chr>    <chr> <chr> <dbl> <chr>    <chr>    <int>    <int> <dbl>    <dbl>
## 1 Afghanis~ AF   AFG   1980 sp      m         0      14    NA      NA
## 2 Afghanis~ AF   AFG   1980 sp      m        15     24    NA      NA
## 3 Afghanis~ AF   AFG   1980 sp      m        25     34    NA      NA
## 4 Afghanis~ AF   AFG   1980 sp      m        35     44    NA      NA
## 5 Afghanis~ AF   AFG   1980 sp      m        45     54    NA      NA
## 6 Afghanis~ AF   AFG   1980 sp      m        55     64    NA      NA
## 7 Afghanis~ AF   AFG   1980 sp      m        65     NA    NA      NA
## 8 Afghanis~ AF   AFG   1980 sp      f         0      14    NA      NA
## 9 Afghanis~ AF   AFG   1980 sp      f        15     24    NA      NA
## 10 Afghanis~ AF   AFG   1980 sp      f        25     34    NA      NA
## # i 304,070 more rows
```

- 4) Since we ignored the 65+ group in the previous problem we will fix it here. If you examine the data you will notice that 65 was placed into the max_age column and there is no value for min_age for those records. To fix this use mutate() in order to replace the blank value in the min_age column with the value from the max_age column and another mutate to replace the 65 in the max column with an Inf. Be sure to keep the variables as character vectors.

```
#now for the values of 65 in age we need to split them up using mutate and
#ifelse in a nested function. I used a different approach because
#the min_age already had 65 due to my last chunk and in this case I can
#use one mutate and ifelse to add 'Inf' into max age if 65 was present in
#min_age
```

```
testwho <- testwho %>%
  mutate(max_age = ifelse(min_age == "65", "Inf", max_age))

testwho
```

```
## # A tibble: 304,080 x 10
##   country iso2 iso3 year diagnosis gender min_age max_age count population
```

```
##      <chr>      <chr> <chr> <dbl> <chr>      <chr>      <int> <chr>      <dbl>      <dbl>
## 1 Afghanis~ AF      AFG      1980 sp      m              0 14      NA      NA
## 2 Afghanis~ AF      AFG      1980 sp      m             15 24      NA      NA
## 3 Afghanis~ AF      AFG      1980 sp      m             25 34      NA      NA
## 4 Afghanis~ AF      AFG      1980 sp      m             35 44      NA      NA
## 5 Afghanis~ AF      AFG      1980 sp      m             45 54      NA      NA
## 6 Afghanis~ AF      AFG      1980 sp      m             55 64      NA      NA
## 7 Afghanis~ AF      AFG      1980 sp      m             65 Inf      NA      NA
## 8 Afghanis~ AF      AFG      1980 sp      f              0 14      NA      NA
## 9 Afghanis~ AF      AFG      1980 sp      f             15 24      NA      NA
## 10 Afghanis~ AF      AFG      1980 sp      f             25 34      NA      NA
## # i 304,070 more rows
```

5) Find the count per diagnosis for males and females.

See `?sum` for a hint on resolving NA values.

```
#we can group the two variables using group by and plug it into a tibble
#and using pipe functions we can sum up the count and na.rm should drop
#all the NA values
```

```
counttest <- testwho %>%
  group_by(diagnosis, gender) %>%
  summarize(count = sum(count, na.rm = TRUE))
```

```
## `summarise()` has grouped output by 'diagnosis'. You can override using the
## `.groups` argument.
```

```
counttest
```

```
## # A tibble: 6 x 3
## # Groups:   diagnosis [3]
##   diagnosis gender    count
##   <chr>      <chr>    <dbl>
## 1 ep        f        941880
## 2 ep        m       1044299
## 3 sn        f       2439139
## 4 sn        m       3840388
## 5 sp        f      11324409
## 6 sp        m      20586831
```

6) Now create a plot using ggplot and geom_col where your x axis is gender, your y axis represents the counts, and facet by diagnosis. Be sure to give your plot a title and resolve the axis labels.

```
#now using our counttest table that we created we can use ggplot to
#create 3 bar plots to visualize the 3 diagnosis
```

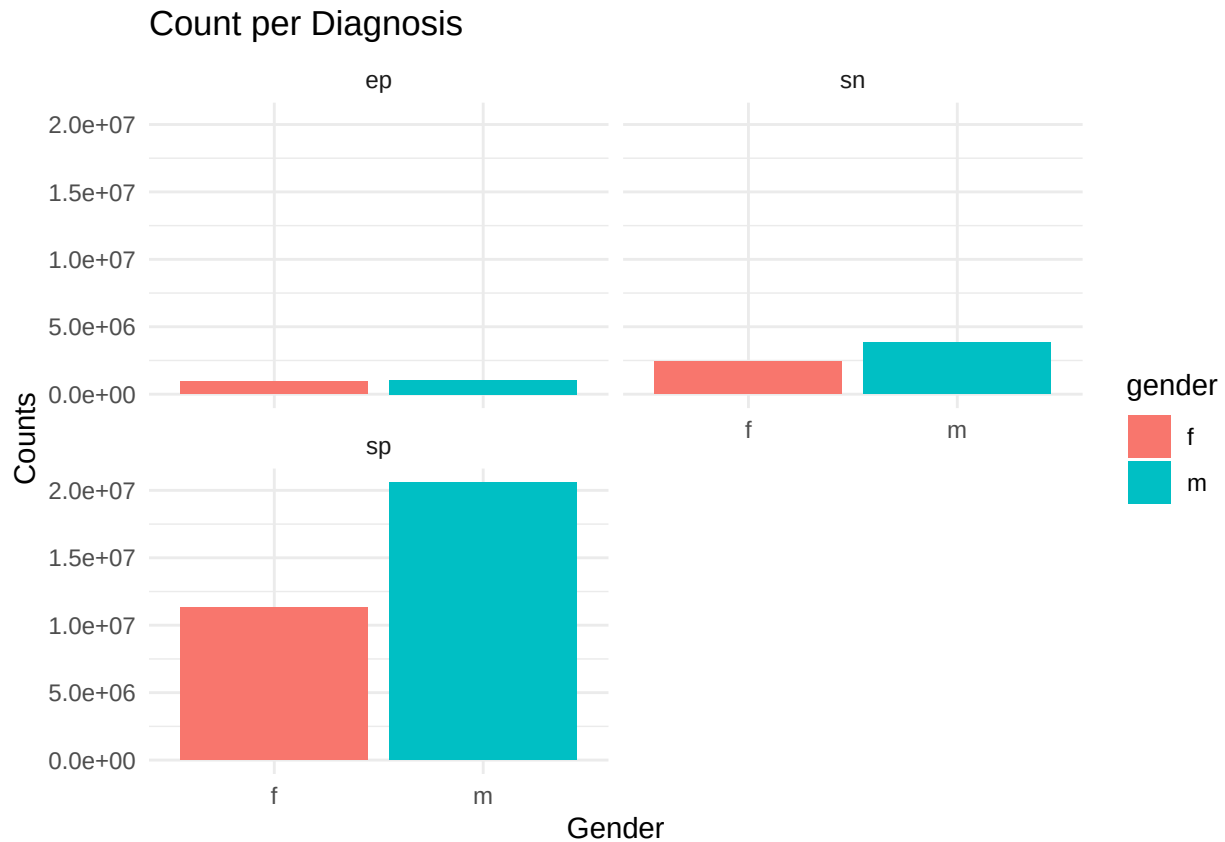
```
ggplot(counttest, aes(x = gender, y = count, fill = gender)) +
  #these are the basic elements such as x and y values, and the fill

  geom_col(position = "dodge") +
  #dodge makes the bars not overlap

  facet_wrap(~diagnosis, ncol = 2) +
  #facet_wrap makes sure we separate the categorical variable and thus
  #group observations by them

  labs(title = "Count per Diagnosis",
```

```
x = "Gender",
y = "Counts") +
theme_minimal()
```



#and these are simple labels

- 7) Find the percentage of population by year, gender, and diagnosis. Be sure to remove rows containing NA values.

```
colnames(testwho)
```

```
## [1] "country" "iso2" "iso3" "year" "diagnosis"
## [6] "gender" "min_age" "max_age" "count" "population"
```

#year, gender, diagnosis

*#I could not tell if the output was correct or not but looking over
#the data set over and over gain i realized that the population
#values started at 1995 and the scale of the count vs the population
was very different thus the percentage value being very low.*

```
per_pop <- testwho %>%
  #first lets drop all the na values so we dont count them towards
  #our calculation
  drop_na() %>%
  group_by(year, gender, diagnosis) %>%
```

```

#we then group the year, gender and diagnosis

summarize(percentage = sum(count) / sum(population) * 100)

## `summarise()` has grouped output by 'year', 'gender'. You can override using
## the `.groups` argument.

#and now using summarize, we can create a percentage column,
#whos values will be the sum of the count/sum of the population *100

per_pop

```

```

## # A tibble: 92 x 4
## # Groups:   year, gender [36]
##   year gender diagnosis percentage
##   <dbl> <chr> <chr>         <dbl>
## 1 1995 f     sp           0.000574
## 2 1995 m     sp           0.000982
## 3 1996 f     sp           0.000663
## 4 1996 m     sp           0.00115
## 5 1997 f     sp           0.000737
## 6 1997 m     sp           0.00131
## 7 1998 f     sp           0.000807
## 8 1998 m     sp           0.00139
## 9 1999 f     ep           0.000138
## 10 1999 f     sn           0.000188
## # i 82 more rows

```

- 8) Create a line plot in ggplot where your x axis contains the year and y axis contains the percent of world population. Facet this plot by diagnosis with each plot stacked vertically. You should have a line for each gender within each facet. Be sure to format your y axis and give your plot a title.

```

#for this we can use a simple ggplot to visualize year and percentage

ggplot(per_pop, aes(x = year, y = percentage, group = gender, color = gender)) +
  #geom_line will produce a line plot
  geom_line() +

  #facet+wrap will group all the observations by the categorical variable
  #diagnosis and make 3 graphs per line
  facet_wrap(~diagnosis, ncol = 3) +

  #simple title and x y elements
  labs(title = "Percentage of World Population by Year and Diagnosis",
        x = "Year",
        y = "World Population (%)") +
  theme_minimal()

```

Percentage of World Population by Year and Diagnosis



9) Now unite the min and max age variables into a new variable named age_range. Use a '-' as the separator.

```
#using unite, we can unite min_age and max_age and plug them back into testwho
testwho <- testwho %>%
  unite(age_range, min_age, max_age, sep = "-")
```

10) Find the percentage contribution of each age group by diagnosis. You will first need to find the count of all diagnoses then find the count of all diagnoses by age group. Join the former to the later and calculate the percent of each age group. Plot these as a geom_col where the x axis is the diagnosis, y axis is the percent of total, and faceted by age group.

```
#using the way you suggested that is exactly what I did

#first lets count all of the diagnosis grouped using summarize
#and put it into a dataset

#all the na values are omitted now
finaldiag <- testwho %>%
  group_by(diagnosis) %>%
  summarize(total_count = sum(count, na.rm = TRUE))

#now we can calculate count of all diagnoses by age group and
#plug them into a dataset

#all the na values are omitted now
```

```

fin_age <- testwho %>%
  group_by(diagnosis, age_range) %>%
  summarize(count = sum(count, na.rm = TRUE))

## `summarise()` has grouped output by 'diagnosis'. You can override using the
## `.groups` argument.

#we can now join the two data set using the same element
#which is the diagnosis variable and then we can mutate them
#to create the percentage we mutate the calculated value onto
#our fin_age table which we can use to plot our bar plot
fin_age <- fin_age %>%
  left_join(finaldiag, by = "diagnosis") %>%
  mutate(percentage = (count / total_count) * 100)

#now we create a barplot using ggplot

ggplot(fin_age, aes(x = diagnosis, y = percentage, fill = age_range)) +
  #we want our data to be seperated by age range and groupped
#together so we used facet_wrap and fill is also age_range
#to show clear difference between them

  geom_col(position = "dodge") +
  #position is dodge so the bars are not overlapping

  facet_wrap(~age_range, ncol = 2, scales = "free_y") +
  #scale is free-y so all the axis labels for y are different
  #scale
  labs(title = "Percentage Contribution of Each Age Group by Diagnosis",
        x = "Diagnosis",
        y = "Percent of Total (%)")

```

Percentage Contribution of Each Age Group by Diagnosis

